Text Analysis

This example shows a few text analyis methods: similarity, clustering, topic modeling and sentiment analysis.

Data

We are providing the data as a list of documents.

```
d1 = "He is a good guy, he is not bad"
d2 = "feet wolves cooked boys girls ,!<@!"
d3 = "He is not a good guy, he is bad"
d4 = "I drink water in parties"
d5 = "I grab a drink in parties"

c3 = [d4, d5]
```

Similarity Measures

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import * # Jaccard
from sklearn.metrics.pairwise import * # Cosine
```

CountVectorizer

`fit()` finds BOW (Bag of Words) and generate vocabulary (indexed alphabetically)

```
vectorizer5 = CountVectorizer()
vectorizer5.fit(c3)
print(vectorizer5.vocabulary_)
```

```
{'drink': 0, 'water': 4, 'in': 2, 'parties': 3, 'grab': 1}
```

`transform()` converts each word to a vector spanned by the vocabulary

```
v_c3 = vectorizer5.transform(c3).toarray()
print(v_c3)
```

```
[[1 0 1 1 1]
 [1 1 1 1 0]]
```

Similarity - cosine_similarity needs input as lists. We can calculate Jaccard similarity because the DTM is binary. (We could have forced binary DTM by using vectorizer5 = CountVectorizer(binary=True)

```
cosine_similarity([v_c3[0]], [v_c3[1]])
```

```
array([[0.75]])
```

```
jaccard_score(v_c3[0],v_c3[1])
```

```
np.float64(0.6)
```

Clustering Analysis

We use K-Means and Agglomerative Clustering for clustering analysis.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
```

```
d6 = "Seattle weather is bad in winter"
d7 = "Seattle Seahawks is a great football team"
d8 = "I love Seahawks"
d9 = "I learned a lot of Data analytics tools"
d10 = "I am a data scientist"
c4 = [d1,d2,d3,d4,d5,d6,d7,d8,d9,d10]
```

```
vectorizer6 = TfidfVectorizer(stop_words='english')
X = vectorizer6.fit_transform(c4)
```

K-Means

`fit()` to train the model which gives cluster_centers_

```
k = 4
model1 = KMeans(n_clusters=k, random_state=12, n_init=10) # random seed & number of times we start with different initial centroids
```

```
model1.fit(X)
```

```
KMeans
KMeans(n_clusters=4, n_init=10, random_state=12)
```

```
model1.cluster_centers_
```

```
array([[0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.23714772, 0.        , 0.        ,
        0.        , 0.23714772, 0.        , 0.        , 0.        ,
        0.38095248, 0.        , 0.        , 0.52544164, 0.20159722,
        0.23714772, 0.        , 0.        , 0.        , 0.        ,
        0.        ],
       [0.        , 0.48770715, 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.40089287,
        0.        , 0.        , 0.40089287, 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.15656201,
        0.        , 0.        , 0.        , 0.18417081, 0.18417081,
        0.        ],
       [0.        , 0.        , 0.1490712 , 0.1490712 , 0.        ,
        0.36241597, 0.1490712 , 0.        , 0.1490712 , 0.        ,
        0.21316295, 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.36241597, 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.21316295, 0.        , 0.        ,
        0.1490712 ],
       [0.23007895, 0.        , 0.        , 0.        , 0.51943254,
        0.        , 0.        , 0.        , 0.        , 0.        ,
```

```
       0.        , 0.        , 0.        , 0.23007895, 0.23007895,
       0.        , 0.        , 0.38095248, 0.        , 0.        ,
       0.        , 0.23007895, 0.        , 0.        , 0.        ,
       0.        ]])
```

`transform()` assigns cluster membership as given by labels_

```
model1.transform(X)
model1clusters = model1.labels_.tolist()
model1clusters
```

```
[1, 2, 1, 2, 2, 1, 0, 0, 3, 3]
```

Characteristics of each cluster

We want to find **5 most significant words for each cluster**.

1. for each cluster, we get the indices of the sorted array in descending order. `argsort()` sorts the array in ascending order and return the indices of sorted array. The index slicing `[:,::-1]` reads the sorted array from the end and effectively re-arranges it in descending order.
2. `get_feature_names_out()` returns the words corresponding to indices.

```
order_centroids = model1.cluster_centers_.argsort()[:,::-1] # order ascending and then reverse it for each cluster
```

```
terms = vectorizer6.get_feature_names_out() # the tokens or terms
```

```
for c in range(k): # for each cluster
  print(f'Cluster {c}:') # print cluster and its number
  for ind in order_centroids[c,:4]: # for the top 4 terms of each centroid
    print(terms[ind])
```

```
Cluster 0:
seahawks
love
great
football
Cluster 1:
bad
guy
good
winter
Cluster 2:
parties
drink
grab
water
Cluster 3:
data
scientist
learned
analytics
```

Agglomerative Clustering

1. **n_clusters** - number of clusters pre-selected
2. **affinity** - distance measure between documents, others are: manhattan, cosince
3. **linkage** - distance measure between clusters, others are: single, complete, average

`fit_predict()` fits the hierarchical clustering from features or distance matrix, and returns cluster labels.

```
from sklearn.cluster import AgglomerativeClustering
```

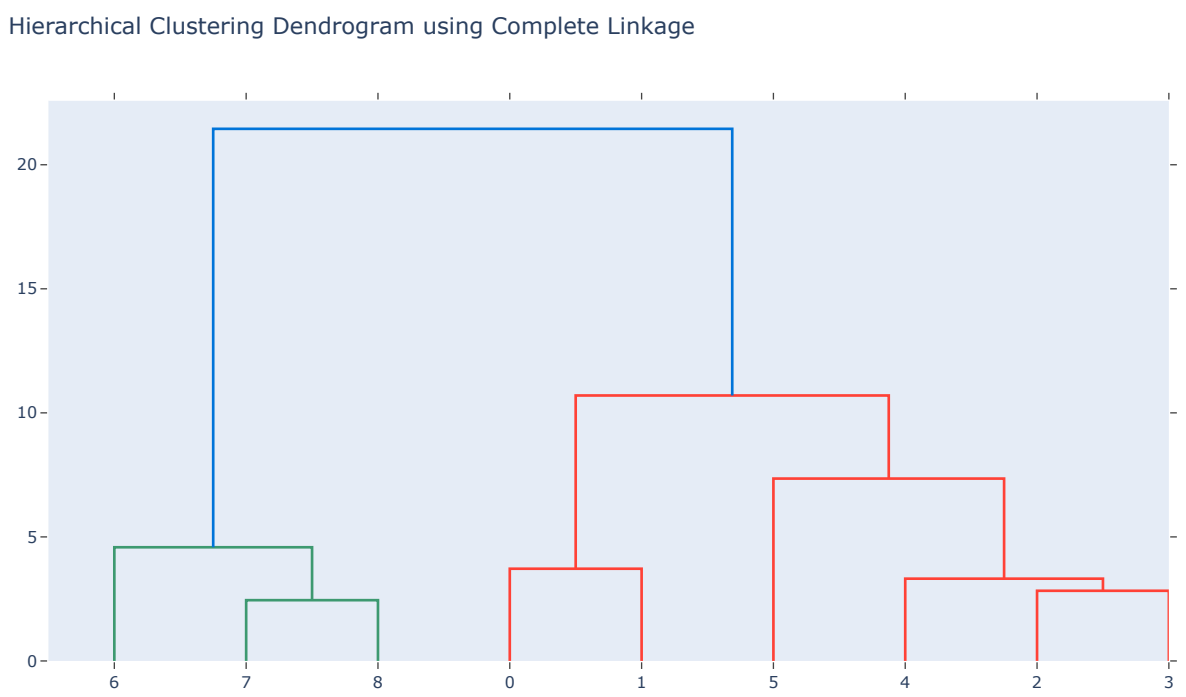
```
# complete linkage you pick two furthest points within two clusters to measure the distance
model2 = AgglomerativeClustering(n_clusters=4, metric='euclidean', linkage='complete')
```

```
model2.fit_predict(X.toarray())
print(model2.labels_)
```

```
[1 0 1 0 0 1 2 2 3 3]
```

```
## Packages to plot dendrogram
from scipy.cluster.hierarchy import dendrogram, linkage
import plotly.figure_factory as ff
```

```
Z = linkage(X.toarray(), method="complete")
fig_complete = ff.create_dendrogram(Z, orientation='bottom')
fig_complete.update_layout(width=1000, height=600, title='Hierarchical Clustering Dendrogram using Complete Linkage')
fig_complete.show()
```



LDA Topic Modeling

Data and document vectors

```
from sklearn.feature_extraction.text import CountVectorizer # for lda, we normally don't use TF-IDF
```

```
vectorizer7 = CountVectorizer(stop_words='english')
```

```
X2 = vectorizer7.fit_transform(c4)
terms = vectorizer7.get_feature_names_out() # similar to .vocabulary_ but without the indices and sorted by index
```

```
terms
```

```
array(['analytics', 'bad', 'boys', 'cooked', 'data', 'drink', 'feet',
       'football', 'girls', 'good', 'grab', 'great', 'guy', 'learned',
       'lot', 'love', 'parties', 'scientist', 'seahawks', 'seattle',
       'team', 'tools', 'water', 'weather', 'winter', 'wolves'],
      dtype=object)
```

Latent Dirichlet Allocation

- **n_components** - number of topics
- The model **lda**'s attribute **components_** stores topic word distribution. The array **components_[i, j]** can be viewed as pseudocount that represents the number of times **word j** was assigned to **topic i**.

To display the representative words under each topic, for each topic:

1. sort the indices of words in the descending order of pseudocount
2. take the top 4
3. retrieve the corresponding words and join them with space " " inbetween them

```
from sklearn.decomposition import LatentDirichletAllocation
lda = LatentDirichletAllocation(n_components=4, random_state=12).fit(X2)
```

```
for topic_idx, topic in enumerate(lda.components_):
  print(f'Topic {topic_idx}:')
  print(" ".join(terms[i] for i in topic.argsort()[:-5:-1])) # elements starting from the end of the sequence, moving backwards, up to (but not including) the 5th element from the end
```

```
Topic 0:
seahawks seattle team great
Topic 1:
data tools learned analytics
Topic 2:
parties drink boys feet
Topic 3:
bad guy good grab
```

Probability of a document containing a topic

```
lda.transform(X2[8])
```

```
array([[0.0417308 , 0.87435871, 0.04220232, 0.04170817]])
```

LDA using gensim

Prepare the data

list of lists of words from list of documents

```
import nltk
from nltk.corpus import stopwords
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
## Preprocessing the documents for gensim
lemmatizer = nltk.stem.WordNetLemmatizer()
stemmer = nltk.stem.PorterStemmer()

processed_c4 = []
for doc in c4:
  tokens = nltk.word_tokenize(doc.lower())
  tokens = [lemmatizer.lemmatize(token) for token in tokens if token.isalpha()]
  tokens = [stemmer.stem(token) for token in tokens]
  tokens = [token for token in tokens if not token in stopwords.words('english')]
  processed_c4.append(tokens)

print(processed_c4)
```

```
[['good', 'guy', 'bad'], ['foot', 'wolf', 'cook', 'boy', 'girl'], ['good', 'guy', 'bad'], ['drink', 'water', 'parti'], ['grab', 'drink', 'parti'], ['seattl', 'weather', 'bad', 'winter']
```

gensim

In terminal, **pip install gensim**

1. create the **dictionary** which is a list of words from all documents
2. convert document into the bag-of-words format = list of (word_id, word_count) 2-tuples
3. train the model with number of topics **num_topics** and mapping from word IDs to words **id2word**
4. use `print_topics(num_topics, num_words)` to display **num_topics** randomly selected topics with a string of **num_words** words ordered by their significance. Both are optional with the default `num_topics=20, num_words=10`. `num_topics = -1` means to show all topics.

```
!pip install gensim
import gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.0.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 33.2 MB/s eta 0:00:00
```

```
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
dictionary = gensim.corpora.Dictionary(processed_c4)
bow_c4 = [dictionary.doc2bow(doc) for doc in processed_c4]
print(bow_c4)
```

```
[[(0, 1), (1, 1), (2, 1)], [(3, 1), (4, 1), (5, 1), (6, 1), (7, 1)], [(0, 1), (1, 1), (2, 1)], [(8, 1), (9, 1), (10, 1)], [(8, 1), (9, 1), (11, 1)], [(0, 1), (12, 1), (13, 1), (14, 1)],
```

```
lda_model = gensim.models.LdaModel(bow_c4, num_topics=4, id2word=dictionary
                                   ,passes=10
                                   ,iterations=200
                                   ,random_state=12)
```

```
for idx, topic in lda_model.print_topics(-1,4):
  print(f'Topic {idx}: \n Words: {topic}')
```

```
Topic 0:
 Words: 0.120*"bad" + 0.119*"weather" + 0.119*"winter" + 0.119*"seattl"
Topic 1:
 Words: 0.096*"good" + 0.096*"parti" + 0.096*"drink" + 0.096*"guy"
Topic 2:
 Words: 0.145*"data" + 0.081*"learn" + 0.081*"analyt" + 0.081*"lot"
Topic 3:
 Words: 0.110*"seahawk" + 0.109*"seattl" + 0.109*"footbal" + 0.109*"team"
```

Sentiment Analysis

Install vader_lexicon

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import pandas as pd
nltk.download('vader_lexicon')
```

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
True
```

Data

```
sentences = ["They are smart, cute, and funny.",  # positive sentence example
    "They are smart, cute, and funny!", # punctuation emphasis handled correctly (sentiment intensity adjusted)
    "They are very smart, cute, and funny.",# booster words handled correctly (sentiment intensity adjusted)
    "They are VERY SMART, cute, and FUNNY.",  # emphasis for ALLCAPS handled
    "They are VERY SMART, cute, and FUNNY!!!",  # combination of signals - VADER appropriately adjusts intensity
    "They are VERY SMART, really handsome, and INCREDIBLY FUNNY!!!",  # booster words & punctuation make this close to ceiling for score
    "The book was good.",  # positive sentence
    "The book was kind of good.",  # qualified positive sentence is handled correctly (intensity adjusted)
    "The plot was good, but the characters are uncompelling and the dialog is not great.",  # mixed negation sentence
    "A really bad, horrible book.",  # negative sentence with booster words
    "At least it isn't a horrible book.",  # negated negative sentence with contraction
    ":) and :D",  # emoticons handled
    "",  # an empty string is correctly handled
    "Today sux",  # negative slang handled
    "Today sux!",  # negative slang with punctuation emphasis handled
    "Today SUX!",  # negative slang with capitalization emphasis
    "Today kinda sux! But I'll get by, lol"  # mixed sentiment example with slang and contrastive conjunction "but"
     ]
```

Sentiment analysis with existing classifier

1. use `SentimentIntensityAnalyzer()`
2. get polarity scores: compound, neg, neu, pos. The output is a dictionary data type which is joined by a dictionary of sentence using `update()`.
3. convert list of dictionary to `DataFrame` and show.

```
sid = SentimentIntensityAnalyzer()
```

```
scores = []
for sentence in sentences:
  score = {'sentence': sentence}
  score.update(sid.polarity_scores(sentence))
  scores.append(score)

df_scores = pd.DataFrame(scores)
df_scores
```

	sentence	neg	neu	pos	compound
0	They are smart, cute, and funny.	0.000	0.259	0.741	0.8225
1	They are smart, cute, and funny!	0.000	0.252	0.748	0.8356
2	They are very smart, cute, and funny.	0.000	0.304	0.696	0.8470
3	They are VERY SMART, cute, and FUNNY.	0.000	0.249	0.751	0.9196
4	They are VERY SMART, cute, and FUNNY!!!	0.000	0.236	0.764	0.9318
5	They are VERY SMART, really handsome, and INCR...	0.000	0.294	0.706	0.9469
6	The book was good.	0.000	0.508	0.492	0.4404
7	The book was kind of good.	0.000	0.657	0.343	0.3832
8	The plot was good, but the characters are unco...	0.327	0.579	0.094	-0.7042
9	A really bad, horrible book.	0.791	0.209	0.000	-0.8211
10	At least it isn't a horrible book.	0.000	0.637	0.363	0.4310
11	:) and :D	0.000	0.124	0.876	0.7925
12		0.000	0.000	0.000	0.0000
13	Today sux	0.714	0.286	0.000	-0.3612
14	Today sux!	0.736	0.264	0.000	-0.4199
15	Today SUX!	0.779	0.221	0.000	-0.5461
16	Today kinda sux! But I'll get by, lol	0.138	0.517	0.344	0.5249